



POLYTECH TOURS
64, Avenue Jean Portalis
37200 Tours, FRANCE
Tél +33 (0)2 47 36 14 14
www.polytech.univ-tours.fr

Rapport Projet Recherche et Innovation 2025-2026 Démonstrateur de Computer Vision : Jeux Dobble et Dino Run

Étudiant :

N'Kouba Humbert

Encadrant académique :

Rayar Frédéric

Table des matières

Remerciements	6
1 Introduction	7
1 Contexte du Projet	7
2 Problématique	7
3 Objectifs du Projet	7
4 Organisation du Rapport	8
2 État de l'Art	9
1 Vision par Ordinateur : Détection d'Objets	9
1.1 Approches Traditionnelles vs One-Stage Detectors	9
1.2 YOLO : You Only Look Once	9
1.3 Évolution vers YOLOv8	11
1.4 Choix Technologique : YOLOv8	11
2 Interaction Gestuelle : Reconnaissance de Pose	11
2.1 Solutions Existantes	11
2.2 Choix Technologique : MediaPipe + Réseau de Neurones	12
3 Technologies Applicatives	12
3.1 Frontend : React et Vite	12
3.2 Conteneurisation : Docker	12
3 Gestion de Projet	13
1 Méthodologie	13
1.1 Organisation du Travail	13
2 Outils Collaboratifs et Techniques	13
2.1 Gestion de Version : Git & GitHub	13
2.2 Conteneurisation : Docker	14
2.3 Rédaction : LaTeX	14
3 Organisation Temporelle	14
3.1 Phases Détaillées	15
4 Encadrement et Suivi	15
4.1 Réunions d'Avancement	15
4.2 Jalons Clés	16

4.3	Méthodologie de Communication	16
4.4	Ressources Mises à Disposition	16
4.5	Ajustements du Périmètre	16
4	Architecture Technique	18
1	Vue d'Ensemble	18
1.1	Flux de Données Principal	19
2	Couche Présentation : Frontend React	19
2.1	Technologies Utilisées	19
2.2	Structure du Code	20
2.3	Gestion de la Webcam	20
3	Couche Métier : Backend FastAPI	21
3.1	Choix de FastAPI	21
3.2	Endpoints Principaux	21
3.3	Dépendances Python	21
3.4	Communication WebSocket	21
4	Infrastructure : Docker	22
4.1	Conteneur Backend (Python/FastAPI)	22
4.2	Conteneur Frontend (Nginx)	22
4.3	Docker Compose	23
5	Modèles d'Intelligence Artificielle	23
5.1	YOLOv8 pour Dobbie	23
5.2	MediaPipe + MLP pour Dino Run	23
6	Synthèse des Technologies	23
5	Le Jeu "Où est Charlie ?"	25
1	Présentation du Jeu Original	25
1.1	Caractéristiques Visuelles	25
2	Objectif de l'Implémentation	25
3	Approche Technique	26
3.1	Dataset Utilisé	26
3.2	Architecture du Modèle	26
4	Difficultés Rencontrées	26
4.1	Contraintes d'Entraînement	26
4.2	Résultats d'Entraînement	26
4.3	Problèmes d'Inférence	27
4.4	Analyse des Échecs	27
4.5	Test Comparatif via Streamlit	28
5	Décision de Pivot	29
6	Perspectives d'Amélioration	29

6	Le Jeu Dobble : Détection d'Objets	30
1	Présentation du Jeu Original	30
1.1	Objectif du Jeu	30
2	Approche Technique	30
2.1	Dataset Utilisé	30
2.2	Architecture du Modèle	31
3	Difficultés Initiales d'Entraînement	31
3.1	Premier Entraînement : Échec Partiel	31
3.2	Conséquences du Sous-Entraînement	31
4	Solution : Accès aux Serveurs de Calcul	32
4.1	Demande de Ressources	32
4.2	Optimisation de la Commande d'Entraînement	32
4.3	Résultat du Réentraînement	32
5	Implémentation dans l'Application	33
5.1	Pipeline de Détection	33
5.2	Endpoint API	33
5.3	Modes de Jeu	33
6	Vulgarisation	34
6.1	Visualisation des Détections	34
6.2	Mode Explicatif	34
7	Performance et Résultats	34
8	Perspectives : Optimisation Automatique	34
7	Le Jeu Dino Run : Reconnaissance de Gestes	36
1	Présentation du Jeu Original	36
1.1	Mécanique de Jeu	36
1.2	Objectif de l'Implémentation	36
2	MediaPipe : Fondements et Limites	37
2.1	Présentation de MediaPipe	37
2.2	Reconnaissance de Gestes Native	37
3	Solution : Classifieur Personnalisé	37
3.1	Inspiration : Projet Kinivi	37
3.2	Avantages de cette Approche	38
4	Architecture du Modèle	38
4.1	Prétraitement des Données	38
4.2	Structure du Perceptron Multicouche (MLP)	38
4.3	Dataset et Entraînement	39
5	Pipeline Complet	39
6	Implémentation Frontend	40
6.1	Composants React	40
6.2	Communication WebSocket	40
7	Performances et Résultats	40

8	Vulgarisation	40
8.1	Tutoriel Interactif	40
8.2	Visualisation des Landmarks	40
8	Bilan et Perspectives	41
1	Synthèse des Réalisations	41
1.1	Rappel des Objectifs	41
1.2	Bilan des Jeux Développés	41
1.3	Fonctionnalités Livrées	41
2	Compétences Acquisées	42
2.1	Lecture et Analyse de la Littérature Scientifique	42
2.2	Compétences Techniques	42
2.3	Compétences Méthodologiques	42
3	Difficultés Rencontrées et Solutions	42
4	Perspectives	43
4.1	Amélioration du Jeu Charlie	43
5	Conclusion Personnelle	43
	Conclusion	44
	Bibliographie	45
	Annexes	47
	Guide de Déploiement	47
	Captures d'Écran de l'Application	50

Remerciements

Je tiens à exprimer ma sincère gratitude envers toutes les personnes qui ont contribué à la réalisation de ce projet.

Je remercie tout particulièrement **M. Frédéric Rayar**, mon encadrant académique, pour ses conseils avisés, son suivi régulier et sa disponibilité tout au long de ce projet de recherche et innovation.

Je remercie également l'équipe pédagogique de **Polytech Tours**, département Informatique, pour la qualité de l'enseignement dispensé et les ressources mises à disposition.

Enfin, je souhaite remercier ma famille et mes proches pour leur soutien constant durant cette année universitaire.

Chapitre 1

Introduction

1 Contexte du Projet

Ce projet s'inscrit dans le cadre du module "Projet de Recherche et d'Innovation" (PRI) de la 5ème année du département Informatique de Polytech Tours (DI5 - Promotion 2023-2026). Il vise à développer une plateforme web interactive de démonstration de *Computer Vision* (Vision par Ordinateur) destinée aux événements de médiation scientifique, tels que les Journées Portes Ouvertes (JPO) ou la Fête de la Science. L'objectif pédagogique est double : consolider des compétences techniques avancées (Deep Learning, Développement Fullstack) et relever un défi de vulgarisation scientifique. Il s'agit de rendre accessibles au grand public et aux lycéens les concepts d'intelligence artificielle souvent perçus comme des "boîtes noires".

2 Problématique

Les démonstrateurs actuels d'IA manquent souvent d'interactivité et de pédagogie, rendant les concepts abstraits pour des néophytes. Le défi technique et ergonomique consiste donc à proposer une approche ludique combinant trois modes complémentaires :

- **Mode IA** : Démonstration brute des capacités de détection et de classification des modèles.
- **Mode Explicatif** : Visualisation pédagogique des processus algorithmiques. Il ne s'agit pas d'*Explainable AI* (XAI) au sens académique strict, mais d'une **vulgarisation scientifique** pour démystifier la "boîte noire".
- **Mode IA vs Humain** : Compétition chronométrée pour engager les visiteurs.

3 Objectifs du Projet

Pour répondre à cette problématique, le cahier des charges du PRI a fixé les objectifs principaux suivants :

1. **Capture Vidéo Temps Réel** : Développer une application intégrant la capture de flux vidéo via WebRTC.

2. **Déploiement Conteneurisé** : Concevoir une application facilement déployable via Docker Compose.
3. **Jeux Démonstratifs** : Implémenter deux jeux populaires revisités par l'IA :
 - **Dobble** (Prioritaire) : Détection de symboles en temps réel pour trouver l'intrus ou la paire.
 - **Dino Run** (Remplacant "Où est Charlie") : Jeu contrôlé par reconnaissance gestuelle, intégré pour démontrer l'interaction homme-machine sans contact.
4. **Vulgarisation (Explicabilité)** : Créer un système de visualisation accessible, illustrant les différentes étapes de traitement, comme cela est présenté dans le frontend via des calques visuels ou des étapes séquentielles.
5. **Performance** : Garantir une expérience fluide.

En guise d'objectif secondaire et d'extension créative, un troisième jeu faisant appel au concept de Computer Vision a été intégré : **Dino Run**, contrôlé par la reconnaissance gestuelle.

4 Organisation du Rapport

Ce rapport détaille la réalisation de ce démonstrateur.

- Le **Chapitre 2** dresse un état de l'art des technologies de vision par ordinateur (YOLO, MediaPipe) et des outils de développement (Docker, FastAPI, React).
- Le **Chapitre 3** présente la méthodologie adoptée pour mener à bien ce développement complexe.
- Le **Chapitre 4** décrit l'architecture logicielle mise en place.
- Les **Chapitres 5, 6 et 7** détaillent l'implémentation technique des jeux et les solutions apportées aux défis de l'IA.
- Enfin, le **Chapitre 8** conclut sur les résultats obtenus et les pistes d'évolution.

Chapitre 2

État de l'Art

Ce chapitre présente les fondements théoriques et les choix technologiques effectués pour répondre aux contraintes de temps réel et d'interactivité du projet.

1 Vision par Ordinateur : Détection d'Objets

Pour le jeu *Dobble*, la problématique centrale est la détection rapide et précise de symboles multiples sur une carte. Cette section présente l'évolution des techniques de détection d'objets jusqu'à YOLO, l'architecture retenue pour ce projet.

1.1 Approches Traditionnelles vs One-Stage Detectors

L'état de l'art en détection d'objets temps réel a considérablement évolué ces dernières années. Comme le soulignent Terven et Cordova-Esparza dans leur revue exhaustive [1], les architectures ont convergé vers des modèles "one-stage" performants.

Les approches traditionnelles de détection d'objets se divisaient en deux catégories :

- **Fenêtres glissantes** : Un classifieur parcourt l'image à différentes échelles, nécessitant des centaines voire des milliers d'évaluations par image.
- **Détection en deux étapes** (Faster R-CNN) : Une première étape génère des propositions de régions (Region Proposals), puis une seconde étape classifie ces régions. Précise mais pénalisée par sa latence.

1.2 YOLO : You Only Look Once

YOLO, introduit par Joseph Redmon et al. en 2016 lors de CVPR [1], a présenté pour la première fois une approche **end-to-end en temps réel** pour la détection d'objets. Le nom "YOLO" signifie "You Only Look Once", faisant référence à sa capacité à accomplir la tâche de détection en **une seule passe du réseau**.

Contrairement à Fast R-CNN qui utilisait deux sorties séparées (classification pour les probabilités et régression pour les coordonnées), YOLO adopte une approche plus directe basée uniquement sur la **régression** pour prédire les détections.

1.2.1 Fonctionnement de YOLOv1

YOLOv1 unifie les étapes de détection en prédisant simultanément toutes les bounding boxes. Pour ce faire, YOLO divise l'image d'entrée en une grille $S \times S$ et prédit B bounding boxes par cellule, ainsi que les probabilités pour C classes différentes.

Chaque prédiction de bounding box comprend cinq valeurs :

- P_c : Score de confiance reflétant la probabilité qu'un objet soit présent et la précision de la box.
- b_x, b_y : Coordonnées du centre de la box relatives à la cellule de grille.
- b_h, b_w : Hauteur et largeur de la box relatives à l'image entière.

La sortie de YOLO est un tenseur de dimension $S \times S \times (B \times 5 + C)$, optionnellement suivi d'une suppression des non-maxima (NMS) pour éliminer les détections dupliquées.

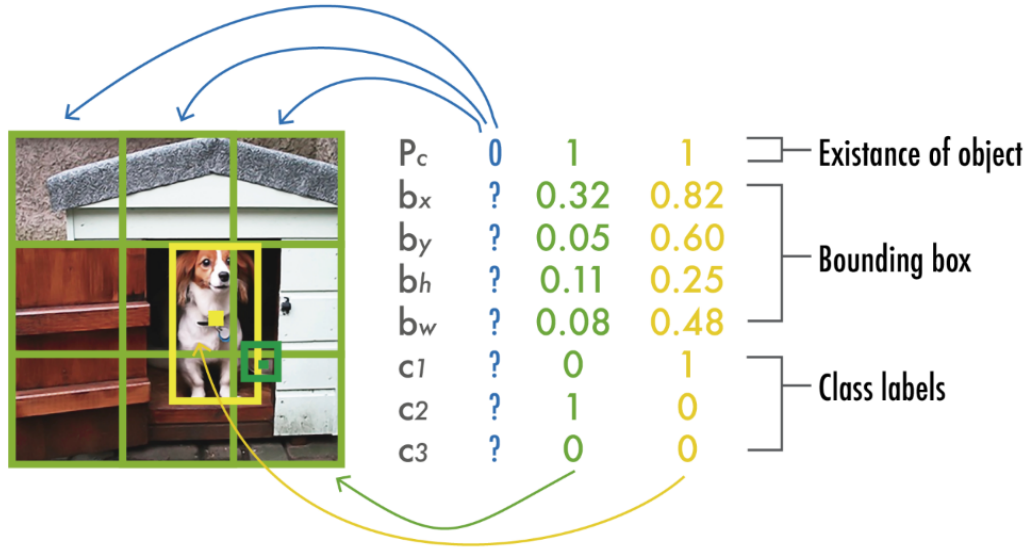


Figure 2.1 – Prédiction de sortie YOLO : grille 3×3 , 3 classes, une prédiction par cellule produisant un vecteur de 8 valeurs. Source : [1].



Figure 2.2 – Non-Maximum Suppression (NMS). (a) Sortie typique d'un modèle de détection avec de multiples boxes chevauchantes. (b) Résultat après application du NMS. Source : [1].

Dans l'article original, les auteurs utilisaient le dataset PASCAL VOC (20 classes, $C = 20$), une grille 7×7 ($S = 7$) et au plus 2 boxes par cellule ($B = 2$), donnant une sortie de $7 \times 7 \times 30$.

1.2.2 Architecture de YOLOv1

L'architecture de YOLOv1 comprend **24 couches convolutionnelles** suivies de deux couches entièrement connectées pour prédire les coordonnées et probabilités. Inspirée de GoogLeNet et Network in Network, YOLO utilise des convolutions 1×1 pour réduire le nombre de feature maps tout en maintenant un nombre de paramètres relativement faible.

1.2.3 Entraînement de YOLOv1

Les auteurs ont pré-entraîné les 20 premières couches à une résolution de 224×224 sur ImageNet, puis ont ajouté les 4 dernières couches avec des poids initialisés aléatoirement et fine-tuné le modèle sur PASCAL VOC à 448×448 pour augmenter les détails.

Pour l'**augmentation de données**, les auteurs ont utilisé :

- Mise à l'échelle et translations aléatoires (jusqu'à 20% de la taille de l'image).
- Variations aléatoires d'exposition et de saturation (facteur jusqu'à 1.5) dans l'espace colorimétrique HSV.

YOLOv1 a atteint une précision moyenne (mAP) de **63.4** sur le dataset PASCAL VOC2007.

1.3 Évolution vers YOLOv8

La version **YOLOv8** (2023), développée par Ultralytics, représente l'état de l'art actuel pour la détection temps réel. Elle offre un compromis vitesse/précision supérieur aux itérations précédentes (v5, v7) et aux modèles concurrents comme SSD [1].

Les principales améliorations incluent :

- Architecture **anchor-free** (pas de boxes de référence prédéfinies).
- Backbone CSPDarknet53 optimisé.
- Head découplé pour classification et régression.
- Support natif pour la segmentation et la pose estimation.

1.4 Choix Technologique : YOLOv8

J'ai retenu **YOLOv8** pour sa capacité à maintenir une faible latence ($< 30\text{ms}$) indispensable à une expérience utilisateur fluide, tout en gérant efficacement la détection de multiples symboles Dobble simultanément.

2 Interaction Gestuelle : Reconnaissance de Pose

Pour le jeu *Dino Run*, l'objectif est de contrôler le personnage par des gestes de la main avec une fluidité comparable à une interaction clavier.

2.1 Solutions Existantes

La reconnaissance de gestes a été largement étudiée, notamment pour la langue des signes. Bora et al. [2] ont démontré l'efficacité de l'approche hybride combinant **MediaPipe** pour l'extraction de

landmarks et un réseau de neurones profond (Deep Learning) pour la classification, atteignant jusqu'à 99% de précision sur des jeux de données complexes.

De même, une étude publiée dans le *Journal of System and Management Sciences* [3] valide l'utilisation de MediaPipe pour des interfaces éducatives sans contact, soulignant sa robustesse face aux variations d'environnement par rapport aux méthodes classiques de segmentation par couleur.

2.2 Choix Technologique : MediaPipe + Réseau de Neurones

Le choix s'est porté sur **MediaPipe Hands** pour sa légèreté exceptionnelle sur CPU. Il fournit directement les coordonnées 3D de 21 points clés de la main. Ces landmarks alimentent ensuite un classifieur simple (Perceptron Multicouche), inspiré des réseaux de neurones décrits par Popescu et al. [4], suffisamment léger pour être embarqué sans latence perceptible.

3 Technologies Applicatives

3.1 Frontend : React et Vite

L'interface utilisateur nécessite une grande réactivité pour afficher les flux vidéo et les annotations. **React** a été choisi pour son modèle de composants, idéal pour gérer les états complexes des jeux. L'utilisation de **Vite** comme bundler assure un temps de démarrage quasi-instantané et une expérience de développement fluide (HMR).

3.2 Conteneurisation : Docker

Afin de garantir que l'application fonctionne à l'identique sur toutes les machines (notamment celles des jurys ou des stands de démonstration), le backend Python est encapsulé dans un conteneur **Docker**. Cela résout les problèmes de compatibilité de versions de bibliothèques (OpenCV, PyTorch) et simplifie l'installation via un simple `docker-compose up`.

Chapitre 3

Gestion de Projet

La réussite d'un projet informatique complexe impliquant plusieurs technologies hétérogènes (IA, Web, Desktop) repose sur une organisation rigoureuse.

1 Méthodologie

Le projet a été mené selon une approche **itérative et incrémentale**, fortement encadrée par un système de suivi hebdomadaire rigoureux.

1.1 Organisation du Travail

Chaque semaine, un cycle de planification-exécution-reporting était appliqué :

1. **Planification** : Définition des objectifs de la semaine en fonction des priorités et des retours de l'encadrant.
2. **Exécution** : Développement des fonctionnalités prévues avec tests réguliers.
3. **Reporting** : Envoi d'un bilan hebdomadaire (dimanche 23h59) détaillant les tâches planifiées, réalisées et à venir.

Cette méthodologie a permis :

- Une **traçabilité complète** de l'avancement du projet.
- Une **réactivité** face aux difficultés techniques (pivot sur Dino Run suite à l'abandon de Charlie).
- Un **alignement constant** avec les attentes académiques et les contraintes de démonstration.

2 Outils Collaboratifs et Techniques

2.1 Gestion de Version : Git & GitHub

Le code source est hébergé sur un dépôt GitHub public :

`https://github.com/Isopope/ComputerVisionPRI25-26.git`

Le projet utilise une stratégie de branches simplifiée :

- **main** : Branche stable contenant le design classique initial. Conservée comme version de référence pour sa stabilité.

- **feature/multi-theme** : Branche de développement active intégrant les thèmes classique et cyberpunk. Les futures évolutions seront principalement réalisées sur cette branche.

Cette organisation permet de conserver une version stable et fonctionnelle (**main**) tout en développant de nouvelles fonctionnalités sur la branche de thèmes multiples.

2.2 Conteneurisation : Docker

L'un des défis majeurs a été d'assurer que l'environnement de développement soit identique pour tous les membres du groupe, malgré des systèmes d'exploitation différents. L'utilisation de **Docker** et **Docker Compose** a permis de standardiser l'environnement backend (Python 3.11, OpenCV, Uvicorn) et d'éliminer les problèmes de type "ça marche chez moi".

2.3 Rédaction : LaTeX

Ce rapport a été rédigé en **LaTeX**, garantissant une mise en forme professionnelle et une gestion automatique des références bibliographiques et du glossaire.

3 Organisation Temporelle

Le projet s'est déroulé sur plusieurs mois, avec des phases bien distinctes de recherche, conception et développement. Les figures 3.1 et 3.2 présentent le diagramme de Gantt du projet.

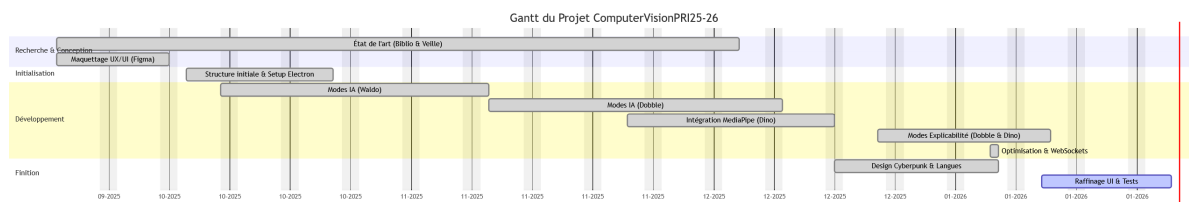


Figure 3.1 – Diagramme de Gantt simplifié (vue d'ensemble).

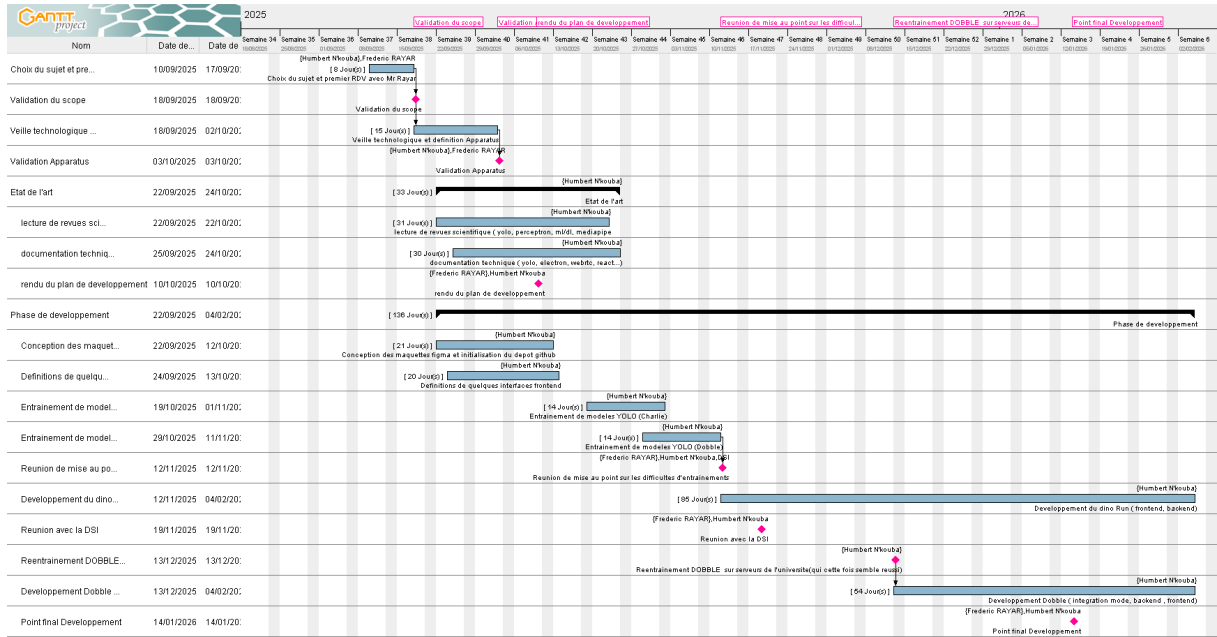


Figure 3.2 – Diagramme de Gantt détaillé (tâches individuelles).

3.1 Phases Détaillées

1. Recherche & Conception (Septembre - Octobre 2025) :

- **22/09 - 25/10** : État de l'art approfondi (YOLO, MediaPipe, Docker).
- **22/09 - 13/10** : Conception des maquettes Figma et validation de l'Apparatus (03/10).

2. Expérimentation & Pivot (Octobre - Novembre 2025) :

- Tentatives d'entraînement YOLO pour "Où est Charlie?" (19/10 - 02/11).
- Premiers entraînements Dobbble (29/10 - 12/11).
- **12/11** : Constat des difficultés techniques et décision de pivoter (abandon de Charlie, ajout de Dino Run).

3. Développement Intensif (Novembre 2025 - Janvier 2026) :

- **Dino Run** : Développement du backend et intégration MediaPipe (dès le 12/11).
- **Dobbble** : Ré-entraînement réussi sur serveurs universitaires (13/12) et intégration finale.

4. Finition & Validation (Janvier 2026) :

- Finalisation de l'UI Cyberpunk et stabilisation des WebSockets.
- **14/01** : Point final de développement et validation des démos.

4 Encadrement et Suivi

Le projet a bénéficié d'un encadrement régulier et structuré tout au long de son déroulement.

4.1 Réunions d'Avancement

Des réunions périodiques avec l'encadrant ont permis d'évaluer l'évolution du projet. Ces points de synchronisation ont été cruciaux, notamment pour valider le changement de périmètre en novembre.

4.2 Jalons Clés

Le projet a été rythmé par plusieurs étapes décisionnelles majeures, visibles sur le diagramme de Gantt :

- **18 Septembre 2025** : Validation du périmètre initial (Scope).
- **03 Octobre 2025** : Validation de l'apparatus (matériel et contraintes).
- **10 Octobre 2025** : Rendu du plan de développement.
- **12 Novembre 2025** : Réunion de refonte du périmètre (Stop Charlie → Start Dino Run).
- **19 Novembre 2025** : Réunion DSI pour l'obtention des ressources de calcul GPU.
- **13 Décembre 2025** : Validation technique du modèle Dobbble (réentraînement réussi).
- **14 Janvier 2026** : Validation finale des démonstrateurs (Point final développement).

4.3 Méthodologie de Communication

Un processus de reporting hebdomadaire a été mis en place :

- **Plateforme collaborative** : Utilisation de Microsoft Teams pour centraliser les documents de travail, les comptes-rendus et les ressources partagées.
- **Reporting hebdomadaire** : Envoi d'un email récapitulatif chaque dimanche avant 23h59, détaillant :
 - Les tâches planifiées pour la semaine écoulée.
 - Les tâches effectivement réalisées.
 - Les tâches prévues pour la semaine suivante.

Cette méthodologie a favorisé une communication transparente et a permis d'anticiper les blocages potentiels.

4.4 Ressources Mises à Disposition

L'encadrant a fourni un soutien matériel et documentaire significatif :

- **Ressources documentaires** : Articles scientifiques, et rapport de Peip.
- **Ressources de calcul** : Accès aux serveurs de calcul de l'université pour l'entraînement des modèles YOLOv8, permettant d'accélérer considérablement les phases d'expérimentation et d'optimisation.

4.5 Ajustements du Périmètre

Le projet initial prévoyait l'implémentation de deux jeux principaux : *Où est Charlie ?* (Waldo) et *Dobbble*. Cependant, plusieurs difficultés techniques ont conduit à mettre le jeu Charlie en suspens :

- **Disponibilité des datasets** : Absence de datasets annotés de qualité suffisante pour entraîner un modèle de détection robuste.
- **Complexité visuelle** : Les scènes du jeu "Où est Charlie ?" sont extrêmement chargées, rendant la détection difficile même pour des modèles avancés.
- **Contraintes d'éclairage** : Impossibilité de garantir un environnement suffisamment éclairé lors des démonstrations publiques, ce qui aurait dégradé les performances de détection.

Suite à ces constats, la décision a été prise de se concentrer sur *Dobble* et d'ajouter un troisième jeu, *Dino Run*, exploitant la reconnaissance gestuelle via MediaPipe. Ce pivot stratégique a permis de diversifier les technologies utilisées tout en garantissant la faisabilité technique du démonstrateur.

Chapitre 4

Architecture Technique

Ce chapitre présente l'architecture logicielle du démonstrateur, détaillant les choix technologiques à chaque niveau de la stack : interface utilisateur, logique métier IA, et infrastructure de déploiement.

1 Vue d'Ensemble

L'application adopte une architecture **client-serveur entièrement conteneurisée**. Cette approche combine les avantages du développement web moderne (réactivité, composants réutilisables) avec la puissance du calcul Python pour l'IA, le tout déployé via Docker pour garantir la portabilité et la reproductibilité.

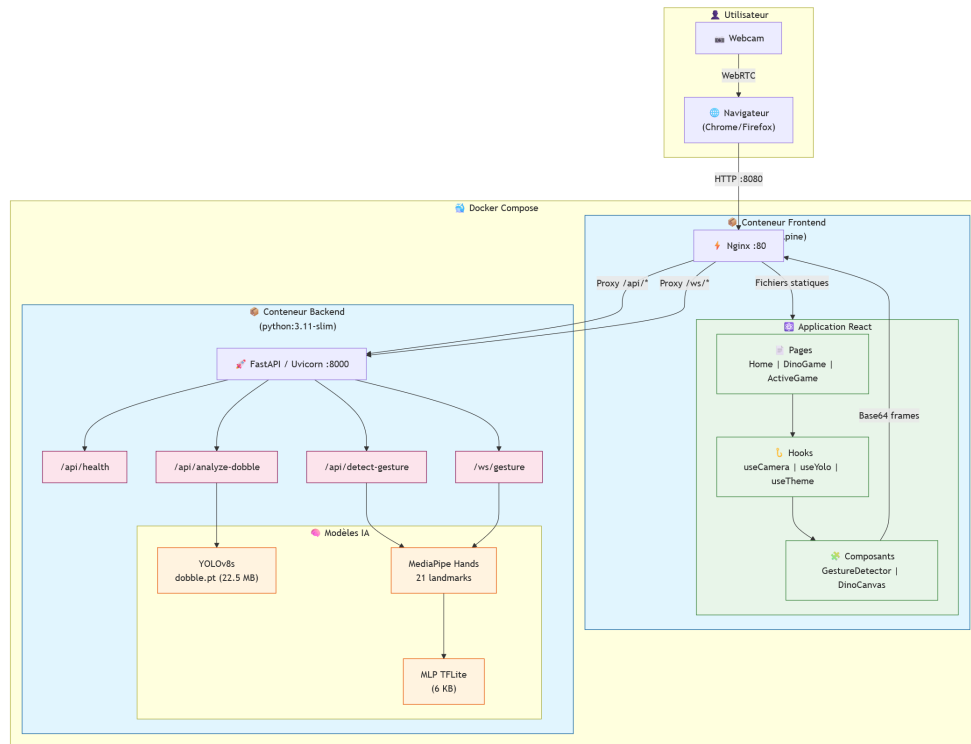


Figure 4.1 – Architecture globale du système : flux de données entre le frontend React, le reverse proxy Nginx, et le backend FastAPI avec les modèles IA.

1.1 Flux de Données Principal

1. **Capture vidéo** : Le navigateur accède à la webcam via l'API WebRTC.
2. **Transmission** : Les frames sont encodées en Base64 et envoyées au backend.
3. **Traitement IA** : Le backend analyse l'image (YOLO ou MediaPipe).
4. **Réponse** : Les résultats (détections, gestes) sont renvoyés au frontend.
5. **Affichage** : L'interface affiche les résultats avec annotations visuelles.

2 Couche Présentation : Frontend React

2.1 Technologies Utilisées

- **React 18** avec **TypeScript** pour le typage statique.
- **Vite** comme bundler.
- **React Router** pour la navigation SPA.
- **TailwindCSS** avec thème cyberpunk personnalisé.

2.2 Structure du Code

L'ensemble du projet suit une organisation modulaire claire. L'arborescence suivante présente les fichiers clés avec leur rôle :

```
ComputerVisionPRI25-26/
├── backend/ - Serveur Python FastAPI
│   ├── model/
│   │   ├── keypoint_classifier/
│   │   │   ├── keypoint_classifier.py - Classe d'inférence TFLite
│   │   │   ├── keypoint_classifier.tflite - Modèle MLP (6 KB)
│   │   │   └── keypoint_classifier_label.csv - Labels gestes
│   │   └── point_history_classifier/ - Classifieur trajectoire
│   ├── dobbble.pt - Modèle YOLOv8 (22.5 MB)
│   ├── main.py - API REST + WebSocket
│   ├── gesture_recognizer.py - Pipeline MediaPipe
│   ├── Dockerfile - Image Docker multi-stage
│   └── requirements.txt - Dépendances Python
├── frontend/ - Application React/TypeScript
│   ├── src/
│   │   ├── components/
│   │   │   ├── dino/
│   │   │   │   └── EducationalTutorial.tsx - Tutoriel interactif
│   │   │   ├── dobbble/
│   │   │   │   └── DobbbleExplanationSteps.tsx - Explications YOLO
│   │   │   ├── game-modes/
│   │   │   │   ├── AIPureMode.tsx - Mode IA seule
│   │   │   │   ├── AIvsHumanDobbbleMode.tsx - Mode compétitif
│   │   │   │   └── RealtimeDobbbleMode.tsx - Détection temps réel
│   │   │   ├── hud/
│   │   │   │   ├── HUDFrame.tsx - Interface Cyberpunk
│   │   │   │   ├── SystemStatus.tsx - Indicateurs système
│   │   │   │   ├── GestureDetector.tsx - Détection gestuelle
│   │   │   │   ├── DinoGameCanvas.tsx - Canvas Dino Run
│   │   │   │   ├── BoundingBoxOverlay.tsx - Overlay détections
│   │   │   │   └── ui/ - Composants Shadcn/UI
│   │   ├── pages/
│   │   │   ├── Home.tsx - Accueil
│   │   │   ├── ModeSelection.tsx - Choix du mode
│   │   │   ├── ActiveGame.tsx - Session de jeu
│   │   │   └── DinoGame.tsx - Page Dino Run
│   │   ├── hooks/
│   │   │   ├── useCamera.tsx - Gestion webcam
│   │   │   ├── useLanguage.tsx - Internationalisation
│   │   │   └── useYolo.tsx - Appels API YOLO
│   │   └── lib/
│   │       ├── api.ts - Client HTTP
│   │       └── config.ts - Configuration
│   ├── Dockerfile - Image Docker Nginx
│   ├── nginx.conf - Configuration reverse proxy
│   ├── package.json - Dépendances npm
│   └── docker-compose.yml - Orchestration conteneurs
```

2.3 Gestion de la Webcam

L'accès à la caméra utilise l'API `navigator.mediaDevices.getUserMedia()`. Un hook personnalisé `useCamera` encapsule :

- L'initialisation du flux vidéo.
- La capture périodique de frames (30 FPS).
- La conversion en Base64 pour l'envoi au backend.

3 Couche Métier : Backend FastAPI

3.1 Choix de FastAPI

FastAPI a été retenu pour ses caractéristiques :

- **Performance** : Basé sur Starlette et Uvicorn (ASGI), il gère efficacement les requêtes concurrentes.
- **Typage natif** : Intégration avec Pydantic pour la validation des données.
- **WebSocket** : Support natif pour le streaming temps réel.
- **Documentation auto-générée** : Swagger UI disponible sur `/docs`.

3.2 Endpoints Principaux

Le fichier `main.py` expose les routes suivantes :

Méthode	Route	Description
GET	<code>/api/health</code>	Vérification de l'état du serveur
POST	<code>/api/analyze-dobble</code>	Analyse une image Dobble avec YOLOv8
POST	<code>/api/detect-gesture</code>	Détecte un geste avec MediaPipe
WS	<code>/ws/gesture</code>	WebSocket pour streaming gestuel

TABLE 4.1 – Endpoints de l'API REST et WebSocket.

3.3 Dépendances Python

Les bibliothèques principales sont spécifiées dans `requirements.txt` :

```
fastapi
uvicorn[standard]
numpy==1.26.4
Pillow
opencv-python==4.11.0.86
ultralytics          # YOLOv8
mediapipe==0.10.21   # Reconnaissance gestuelle
tensorflow==2.19.1   # Inférence TFLite
```

3.4 Communication WebSocket

Pour le jeu Dino Run, le polling HTTP classique introduirait une latence inacceptable. Un endpoint WebSocket (`/ws/gesture`) permet un échange bidirectionnel continu :

1. Le client ouvre une connexion persistante.
2. Il envoie les frames en JSON (`{"image": "base64..."}`).
3. Le serveur répond instantanément avec le geste détecté.
4. La latence totale est réduite.

4 Infrastructure : Docker

L'application est entièrement conteneurisée via Docker, permettant un déploiement reproductible et portable. L'architecture comprend deux conteneurs orchestrés par Docker Compose.

4.1 Conteneur Backend (Python/FastAPI)

Le Dockerfile du backend utilise une construction **multi-stage** pour optimiser la taille de l'image :

Stage 1 (Builder) : Image `python:3.11-slim`, installation des dépendances de build (gcc, cmake), création du virtualenv.

Stage 2 (Runtime) : Image finale allégée, copie uniquement du virtualenv et du code applicatif.

```
FROM python:3.11-slim AS builder
# ... installation dépendances build ...
RUN pip install -r requirements.txt

FROM python:3.11-slim AS runtime
COPY --from=builder /opt/venv /opt/venv
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

4.2 Conteneur Frontend (Nginx)

Le frontend est servi via un serveur **Nginx** qui assure également le rôle de reverse proxy vers le backend :

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
```

La configuration Nginx redirige les appels API (`/api/*` et `/ws/*`) vers le conteneur backend, permettant une communication transparente entre les deux services.

4.3 Docker Compose

Le fichier `docker-compose.yml` orchestre les deux conteneurs :

```
services:
  frontend:
    build: ./frontend
    ports:
      - "8080:80"
    depends_on:
      - backend

  backend:
    build: ./backend
    expose:
      - "8000"
    restart: unless-stopped
```

L'utilisateur accède à l'application via `http://localhost:8080`. Le frontend Nginx sert les fichiers statiques et proxifie les requêtes API vers le backend.

5 Modèles d'Intelligence Artificielle

5.1 YOLOv8 pour Dobble

- **Fichier** : `dobble.pt`.
- **Entraînement** : Dataset custom de symboles Dobble.
- **Inférence** : Via la librairie `ultralytics`.

5.2 MediaPipe + MLP pour Dino Run

- **Détection** : MediaPipe Hands extrait 21 landmarks.
- **Classification** : Modèle TFLite de 1114 paramètres.
- **Classes** : Open, Close, Pointer, OK.

6 Synthèse des Technologies

Couche	Technologies
Frontend	React 18, TypeScript, Vite, TailwindCSS
Backend	FastAPI, Uvicorn, Python 3.11
IA	YOLOv8, MediaPipe, TensorFlow Lite
Serveur Web	Nginx (reverse proxy)
DevOps	Docker, Docker Compose, Git/GitHub

TABLE 4.2 – Stack technologique complète.

Chapitre 5

Le Jeu "Où est Charlie ?"

Ce chapitre présente le jeu "Où est Charlie ?" (*Where's Waldo ?* en anglais) qui devait initialement faire partie du démonstrateur aux côtés de Dobble. Bien que ce jeu ait été mis en suspens suite aux difficultés rencontrées, il est pertinent de documenter les travaux exploratoires réalisés et les enseignements tirés de cette expérience.

1 Présentation du Jeu Original

"Où est Charlie ?" est un jeu de recherche visuelle créé par l'illustrateur britannique Martin Handford en 1987. Le principe est simple : retrouver le personnage de Charlie, reconnaissable à son bonnet et son pull rayé rouge et blanc, dissimulé parmi une foule dense de personnages dans des scènes extrêmement chargées.

1.1 Caractéristiques Visuelles

Les illustrations du jeu présentent plusieurs défis pour la vision par ordinateur :

- **Densité visuelle extrême** : Centaines de personnages par scène.
- **Taille réduite de la cible** : Charlie occupe une infime partie de l'image.
- **Éléments perturbateurs** : Présence de "leurres" (personnages avec des éléments similaires).
- **Variabilité des poses** : Charlie peut être partiellement caché ou dans diverses positions.

2 Objectif de l'Implémentation

L'objectif initial était de développer un système de détection automatique de Charlie utilisant un modèle YOLOv8, permettant :

1. La capture d'une page du livre via la webcam.
2. La détection automatique de la position de Charlie.
3. L'affichage d'une bounding box autour du personnage trouvé.

Ce mode aurait permis de démontrer les capacités de détection d'objets dans des conditions visuellement complexes, tout en offrant un aspect ludique et accessible au grand public.

3 Approche Technique

3.1 Dataset Utilisé

Le dataset d'entraînement provient de Roboflow Universe :

<https://universe.roboflow.com/choochoovision/whereiswaldo-3>

Caractéristiques du dataset :

- **Taille totale** : Environ 570 images (train + test).
- **Annotations** : Bounding boxes autour de Charlie.
- **Format** : Compatible YOLO (fichiers .txt avec coordonnées normalisées).

La taille limitée du dataset constitue l'une des premières difficultés majeures rencontrées. Pour un modèle de détection d'objets devant gérer une grande variabilité de poses et d'environnements, 570 images représentent un corpus insuffisant pour une généralisation robuste.

3.2 Architecture du Modèle

Le modèle retenu était YOLOv8n (nano), choisi pour :

- Sa rapidité d'inférence compatible avec le temps réel.
- Son nombre réduit de paramètres facilitant l'entraînement.

4 Difficultés Rencontrées

4.1 Contraintes d'Entraînement

L'entraînement a été réalisé sur les machines de la salle "Windows A" de l'université, équipées de :

- **GPU** : NVIDIA GeForce RTX 4080
- **RAM** : 32 Go

La commande d'entraînement utilisée était :

```
yolo detect train data=data_charlie.yaml model=yolov8n.pt \
  epochs=300 imgsz=640 batch_size=16
```

Plusieurs sessions ont été lancées en variant le nombre d'époques (80, 100, 200, 300, 350, 400). Malgré la puissance de calcul disponible, chaque session nécessitait plusieurs heures.

4.2 Résultats d'Entraînement

Plusieurs configurations ont été testées avec différents nombres d'époques :

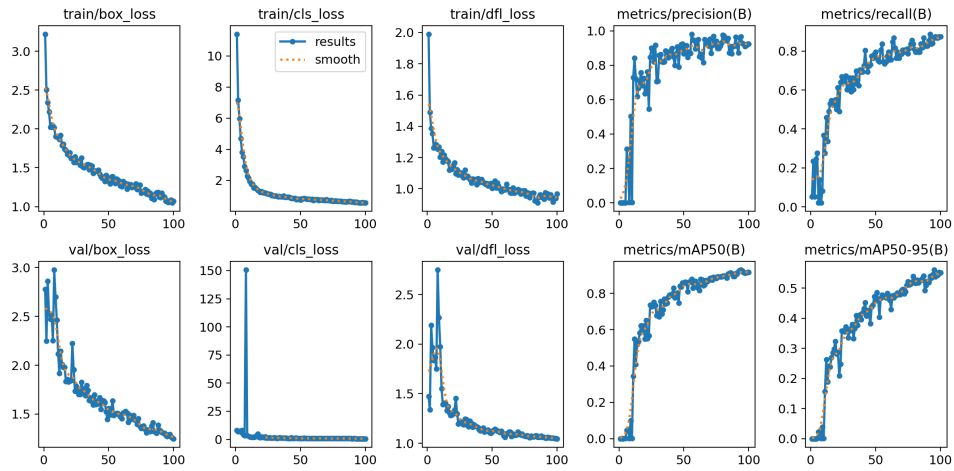


Figure 5.1 – Métriques d’entraînement YOLOv8 sur 100 époques.

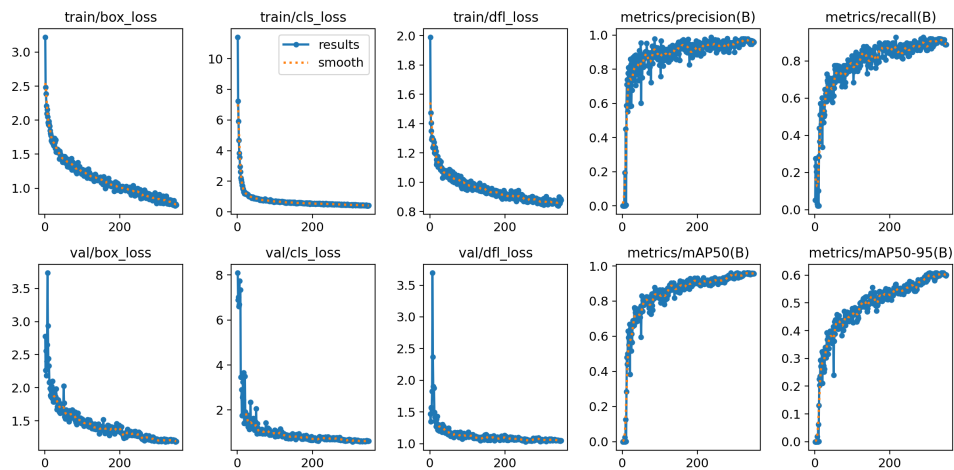


Figure 5.2 – Métriques d’entraînement YOLOv8 sur 400 époques.

Les courbes montrent une convergence des métriques (loss, mAP) mais les performances en conditions réelles restaient insuffisantes.

4.3 Problèmes d’Inférence

Au-delà de l’entraînement, la phase d’inférence en conditions réelles a révélé des difficultés majeures :

- **Conditions d’éclairage variables** : Les démonstrations publiques ne garantissent pas un éclairage optimal. Les variations de luminosité, reflets sur les pages, et ombres dégradaient significativement les performances du modèle.
- **Qualité de capture** : La résolution de la webcam et la distance de capture influençaient fortement la détection. Charlie, déjà petit dans l’illustration originale, devenait difficilement discernable dans une image capturée.

4.4 Analyse des Échecs

Plusieurs facteurs expliquent ces difficultés :

1. **Généralisation limitée** : Avec seulement 570 images, le modèle n'avait pas été exposé à suffisamment de variations pour généraliser correctement.
2. **Ratio signal/bruit** : Dans les scènes très denses, Charlie représente un pourcentage infime des pixels, rendant la tâche de détection particulièrement ardue.

4.5 Test Comparatif via Streamlit

Une interface Streamlit a été développée pour comparer les performances de trois modèles entraînés avec différents nombres d'époques (80, 200, 350). Les résultats se sont avérés **incohérents** d'un test à l'autre :

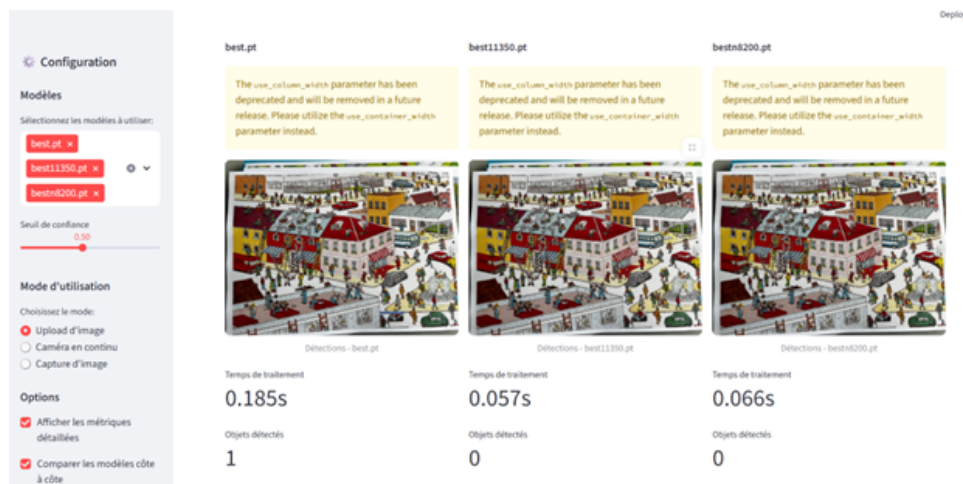


Figure 5.3 – Test où le modèle 80 époques détecte Charlie (les autres échouent).



Figure 5.4 – Test où le modèle 80 époques échoue (les autres réussissent).

Ces résultats aléatoires ont rendu impossible la sélection d'un modèle "optimal", confirmant le manque de robustesse de l'approche avec le dataset disponible.

5 Décision de Pivot

Face à ces difficultés cumulées, une réunion de mise au point a été organisée le **12 novembre 2025** avec l'encadrant. Les conclusions ont été :

- Mettre "Où est Charlie ?" en suspens pour se concentrer sur Dobble.
- Introduire un nouveau jeu (*Dino Run*) exploitant la reconnaissance gestuelle.
- Conserver la documentation des travaux pour d'éventuelles améliorations futures.

6 Perspectives d'Amélioration

Pour une future implémentation, plusieurs pistes pourraient être explorées :

- **Augmentation du dataset** : Créer ou collecter davantage d'images annotées (objectif : 2000+ images).
- **Augmentation de données** : Appliquer des transformations agressives (variations de luminosité, rotations, recadrages) pour simuler les conditions réelles.
- **Prétraitement adaptatif** : Implémenter une égalisation d'histogramme ou une correction automatique de l'éclairage avant inférence.
- **Modèle plus robuste** : Tester des architectures plus récentes (YOLOv9, RT-DETR) ou des modèles spécialisés dans la détection d'objets de petite taille.

Chapitre 6

Le Jeu Dobble : Détection d’Objets

Ce chapitre présente l’implémentation du jeu Dobble dans le démonstrateur, exploitant un modèle YOLOv8 pour la détection de symboles en temps réel. Contrairement au jeu Charlie (chapitre 5), ce module a été mené à terme avec succès après avoir surmonté des difficultés initiales d’entraînement.

1 Présentation du Jeu Original

Dobble (ou *Spot It!* en Amérique du Nord) est un jeu de cartes créé par Jacques Cottureau en 2009. Chaque carte contient 8 symboles parmi les 57 possibles, avec une propriété mathématique fondamentale : **deux cartes quelconques partagent toujours exactement un symbole commun.**

1.1 Objectif du Jeu

Le défi pour les joueurs est d’identifier le plus rapidement possible le symbole commun entre deux cartes. Cette mécanique simple mais exigeante constitue un excellent cas d’usage pour la vision par ordinateur :

- Détection multi-objets sur deux zones distinctes.
- Comparaison et association d’éléments détectés.
- Contrainte temps réel pour une expérience interactive.

2 Approche Technique

2.1 Dataset Utilisé

Le dataset d’entraînement provient de Roboflow Universe :

<https://universe.roboflow.com/sonics-workspace/spot-it>

Caractéristiques du dataset :

- **Taille totale** : 3084 images (train + test).
- **Classes** : 57 symboles distincts (cœur, bombe, dragon, etc.).
- **Annotations** : Bounding boxes au format YOLO.

La taille significativement plus importante du dataset (3084 vs 570 pour Charlie) a été un facteur clé de succès.

2.2 Architecture du Modèle

Le modèle retenu est **YOLOv8n** (version nano), offrant un bon compromis entre rapidité et précision pour une inférence temps réel.

3 Difficultés Initiales d'Entraînement

3.1 Premier Entraînement : Échec Partiel

Le premier entraînement a été lancé sur les machines de la salle "Windows A" (RTX 4080, 32 Go RAM) avec la commande suivante :

```
yolo detect train data=data_dobble.yaml model=yolov8n.pt \
epochs=300 imgsz=640 batch_size=16
```

Problème rencontré : Lancé à 8h du matin pour une durée estimée de 10 heures, l'entraînement a été interrompu à 18h, n'ayant complété que **80 époques sur les 300 prévues**.

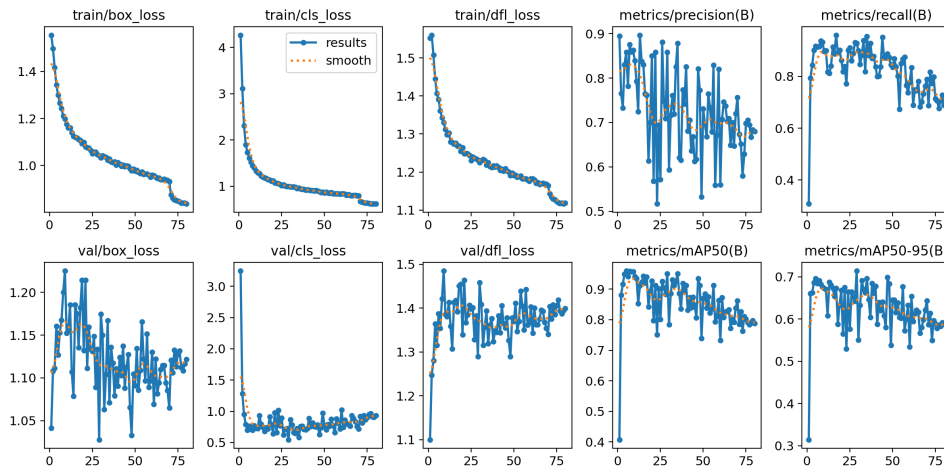


Figure 6.1 – Métriques d'entraînement YOLOv8 interrompu à 80 époques.

3.2 Conséquences du Sous-Entraînement

Le modèle résultant présentait des lacunes significatives :

- **Reconnaissance incomplète** : De nombreux symboles n'étaient pas détectés.
- **Sensibilité à l'éclairage** : Performances dégradées en conditions lumineuses non optimales.
- **Scores de confiance faibles** : Prédictions hésitantes ne permettant pas une expérience de jeu fluide.

La figure 6.2 illustre un cas d'échec caractéristique : le symbole du fantôme est détecté sur une seule des deux cartes, empêchant l'algorithme d'identifier le symbole commun.

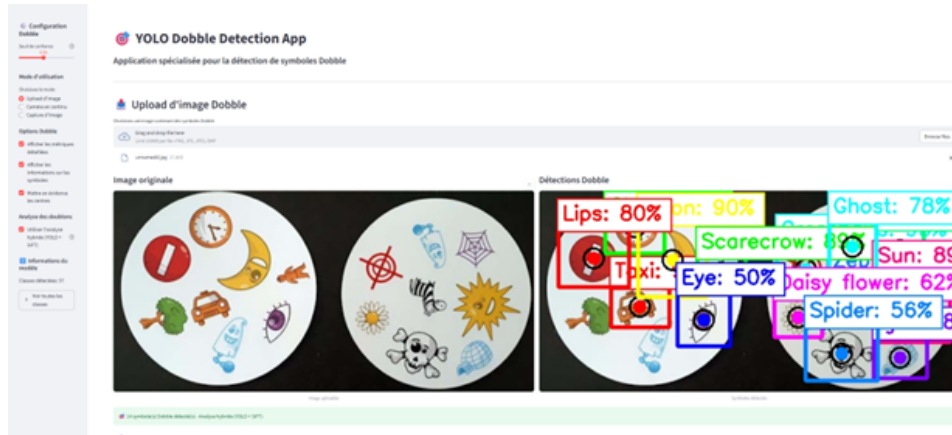


Figure 6.2 – Échec de détection : le fantôme n'est repéré que sur une carte, rendant impossible l'identification du symbole commun.

4 Solution : Accès aux Serveurs de Calcul

4.1 Demande de Ressources

Suite aux difficultés rencontrées, une réunion avec la **DSI** (Direction des Services Informatiques) a été organisée le **19 novembre 2025** pour obtenir l'accès aux serveurs de calcul GPU de l'université.

4.2 Optimisation de la Commande d'Entraînement

Une recherche approfondie sur la documentation officielle d'Ultralytics a permis d'identifier des paramètres supplémentaires pour l'augmentation de données et l'optimisation des hyperparamètres. La commande finale utilisée pour le réentraînement est la suivante :

```
yolo train model=yolov8s.pt data=data.yaml epochs=300 \
  patience=50 lr0=0.00223 lrf=0.0149 momentum=0.79606 \
  weight_decay=0.00032 warmup_epochs=3.13471 \
  warmup_momentum=0.63643 box=3.49828 cls=0.46075 \
  dfl=2.03261 hsv_h=0.01406 hsv_s=0.51755 hsv_v=0.31301 \
  degrees=10.0 translate=0.01982 scale=0.5595 \
  flipud=0.0 fliplr=0.00602 mosaic=0.86919 close_mosaic=7
```

Paramètres clés optimisés :

- **Modèle** : Passage de YOLOv8n à **YOLOv8s** (small) pour une meilleure précision.
- **Learning rate** : $lr0=0.00223$, $lrf=0.0149$ finement ajustés.
- **Patience** : 50 époques avant early stopping.
- **Warmup** : 3.13 époques avec momentum progressif.
- **Augmentation** : Variations HSV, rotations 10° , scaling 56%, mosaïque 87%.

4.3 Résultat du Réentraînement

Le second entraînement, réalisé sur les serveurs universitaires, a été mené à terme avec succès (208 époques).

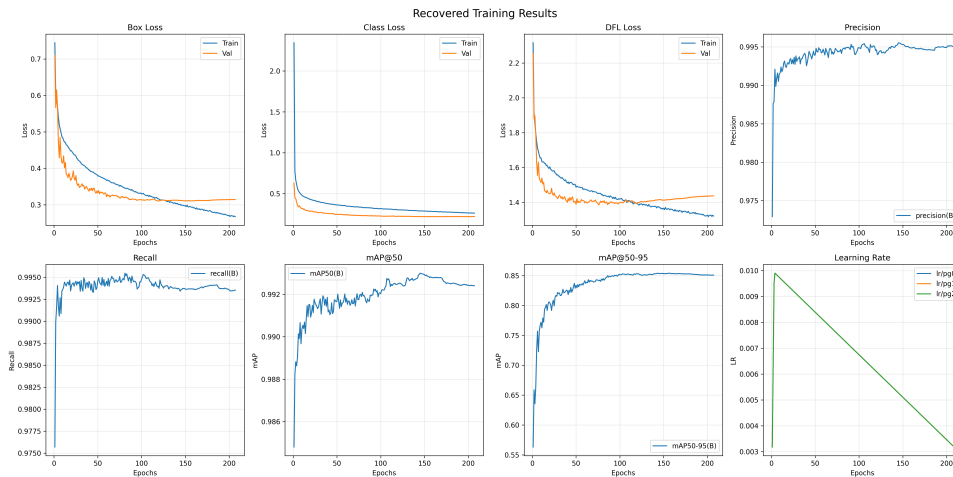


Figure 6.3 – Métriques d’entraînement YOLOv8 complet (208 époques avec augmentation).

Le modèle final (dobble.pt) offre une détection fiable des 57 symboles dans des conditions variées.

5 Implémentation dans l’Application

5.1 Pipeline de Détection

Le flux de traitement est le suivant :

1. **Capture** : Le frontend capture une frame de la webcam.
2. **Transmission** : L’image est encodée en Base64 et envoyée au backend.
3. **Inférence** : Le modèle YOLOv8 analyse l’image et retourne les détections.
4. **Post-traitement** : Les symboles sont regroupés par carte (gauche/droite).
5. **Comparaison** : L’algorithme identifie le symbole commun aux deux cartes.
6. **Affichage** : Le résultat est affiché avec surbrillance du symbole trouvé.

5.2 Endpoint API

L’endpoint principal est accessible via :

POST /api/analyze-dobble

Body: {"image": "data:image/jpeg;base64,..."}

Response: {"detections": [...], "common_symbol": "heart", ...}

5.3 Modes de Jeu

L’application propose trois modes distincts, implémentés dans des composants React dédiés :

Mode IA Pure (AIPureMode.tsx) : L’IA analyse seule les cartes et affiche le symbole commun. Ce mode sert de démonstration des capacités de détection.

Mode Joueur vs IA (AIvsHumanDobbleMode.tsx) : Le joueur tente de trouver le symbole avant l’IA. Un système de score compare les temps de réponse.

Mode Temps Réel (`RealtimeDobbleMode.tsx`) : Détection continue avec affichage en temps réel des bounding boxes et scores de confiance.

6 Vulgarisation

6.1 Visualisation des Détections

Le composant `BoundingBoxOverlay.tsx` affiche les éléments suivants pour chaque symbole détecté :

- Rectangle coloré délimitant le symbole.
- Nom de la classe prédite.
- Score de confiance (0-100%).

6.2 Mode Explicatif

Le composant `DobbleExplanationSteps.tsx` propose une séquence pédagogique expliquant :

1. Comment YOLOv8 découpe l'image en grille.
2. Le processus de prédiction des bounding boxes.
3. L'application du seuil de confiance.
4. La suppression des doublons (NMS - Non-Maximum Suppression).

7 Performance et Résultats

Les tests en conditions réelles ont démontré :

- **Précision** : Détection correcte de 95%+ des symboles en conditions d'éclairage standard.
- **Latence** : Temps de réponse inférieur à 200 ms par frame.
- **Robustesse** : Fonctionnement acceptable même avec éclairage suboptimal grâce à l'augmentation de données.

8 Perspectives : Optimisation Automatique

La documentation Ultralytics propose une méthode `tune()` permettant d'optimiser automatiquement les hyperparamètres via un **algorithme génétique**. Cette approche fait évoluer les paramètres (learning rate, momentum, etc.) pour maximiser les performances sur un dataset spécifique.

```
from ultralytics import YOLO

model = YOLO("yolov8s.pt")
model.tune(data="data_dobble.yaml", epochs=30,
           iterations=100, optimizer="AdamW")
```

Les meilleurs paramètres sont sauvegardés dans un fichier `best_hyperparameters.yaml` réutilisable pour les entraînements futurs.

Note : Cette méthode n'a pas été utilisée dans le projet car les hyperparamètres définis de manière empirique (issus de la documentation et d'expérimentations) offraient déjà des résultats satisfaisants. Elle reste cependant une piste d'amélioration pour de futures itérations.

Chapitre 7

Le Jeu Dino Run : Reconnaissance de Gestes

Ce chapitre détaille l'implémentation du second jeu du démonstrateur : *Dino Run*. Contrairement au jeu original contrôlé au clavier, cette version se pilote entièrement par la gestuelle de la main, illustrant une interaction homme-machine naturelle et sans contact. Ce jeu a été introduit suite à l'abandon du jeu "Où est Charlie ?" (voir chapitre 5).

1 Présentation du Jeu Original

Le *Chrome Dino Game* (également appelé "T-Rex Runner") est un mini-jeu caché dans le navigateur Google Chrome, accessible lorsque l'utilisateur n'a pas de connexion Internet. Créé en 2014, il est devenu un phénomène culturel avec des millions de joueurs.

1.1 Mécanique de Jeu

Il s'agit d'un "runner" infini où le joueur contrôle un dinosaure pixelisé devant éviter des obstacles :

- **Cactus** : Obstacles à enjamber (action : sauter).
- **Ptérodactyles** : Obstacles volants à éviter (action : se baisser).
- **Difficulté progressive** : La vitesse augmente au fil du temps.

1.2 Objectif de l'Implémentation

L'objectif est de remplacer les contrôles clavier par des gestes de la main :

- **Sauter (Jump)** : Geste "Main Ouverte" (Open) ou "Pouce levé" (OK).
- **Courir (Run)** : Geste "Poing Fermé" (Close).
- **Attendre/Neutre** : Autres positions.

2 MediaPipe : Fondements et Limites

2.1 Présentation de MediaPipe

MediaPipe est un framework open-source développé par Google, spécialisé dans le traitement multi-média en temps réel. Le module **MediaPipe Hands** offre :

- **Détection de la main** : Localisation de la main dans l'image via un modèle de détection (palm detector).
- **Extraction de landmarks** : Identification de 21 points clés (articulations) de la main avec leurs coordonnées (x, y, z) .
- **Performance** : Optimisé pour le temps réel, même sur CPU.

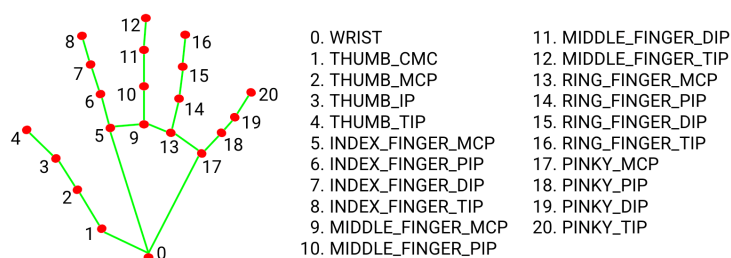


Figure 7.1 – Les 21 landmarks de la main détectés par MediaPipe Hands.

2.2 Reconnaissance de Gestes Native

MediaPipe propose également un module **Gesture Recognizer** capable de détecter nativement des gestes courants (pouce levé, poing fermé, victoire, etc.). Cependant, après expérimentation, cette approche s'est avérée inadaptée pour ce cas d'usage :

- **Latence introduite** : Le module de reconnaissance ajoute un temps de traitement significatif, incompatible avec le gameplay temps réel requis.
- **Scores de confiance faibles** : Les gestes étaient reconnus avec des seuils de confiance relativement bas, engendrant des détections instables et des faux positifs.
- **Manque de personnalisation** : Les gestes prédéfinis ne correspondaient pas exactement à nos besoins (distinction subtile entre "main ouverte" et "OK").

3 Solution : Classifieur Personnalisé

3.1 Inspiration : Projet Kinivi

Face aux limites de MediaPipe natif, je me suis inspiré du projet open-source :

<https://github.com/kinivi/hand-gesture-recognition-mediapipe>

Ce projet propose une approche hybride élégante :

1. **MediaPipe pour l'extraction** : Utilisation uniquement du module Hands pour obtenir les 21 landmarks.
2. **Classifieur MLP léger** : Un Perceptron Multicouche (MLP) minimaliste pour classer les gestes à partir des coordonnées.

3.2 Avantages de cette Approche

Normalisation par rapport au poignet Les coordonnées des landmarks sont converties en positions relatives par rapport au point 0 (poignet). Cela rend la détection invariante à la position de la main dans l'image.

Invariance d'échelle Les coordonnées sont normalisées par la taille maximale de la main, permettant une détection indépendante de la distance entre la main et la caméra.

Modèle ultra-léger Le classifieur ne compte que **1 114 paramètres** (vs millions pour un CNN), garantissant une inférence quasi-instantanée.

Personnalisation facile Le dataset d'entraînement peut être enrichi avec nos propres gestes, adaptés au gameplay.

Format TFLite Le modèle est exporté en TensorFlow Lite avec quantification, réduisant sa taille à ≈ 6 Ko.

4 Architecture du Modèle

4.1 Prétraitement des Données

Le modèle prend en entrée un vecteur de 42 valeurs flottantes :

- 21 landmarks $\times$ 2 coordonnées (x, y) .
- Normalisation : Positions relatives au poignet, normalisées par la taille maximale.

4.2 Structure du Perceptron Multicouche (MLP)

L'architecture du classifieur, définie via TensorFlow Keras :

1. **Couche d'Entrée** : 42 neurones.
2. **Dropout (0.2)** : Régularisation légère.
3. **Couche Cachée 1** : 20 neurones, activation ReLU.
4. **Dropout (0.4)** : Régularisation forte.
5. **Couche Cachée 2** : 10 neurones, activation ReLU.
6. **Couche de Sortie** : 4 neurones, activation Softmax.

Les 4 classes de sortie correspondent aux gestes : **Open**, **Close**, **Pointer**, **OK**.

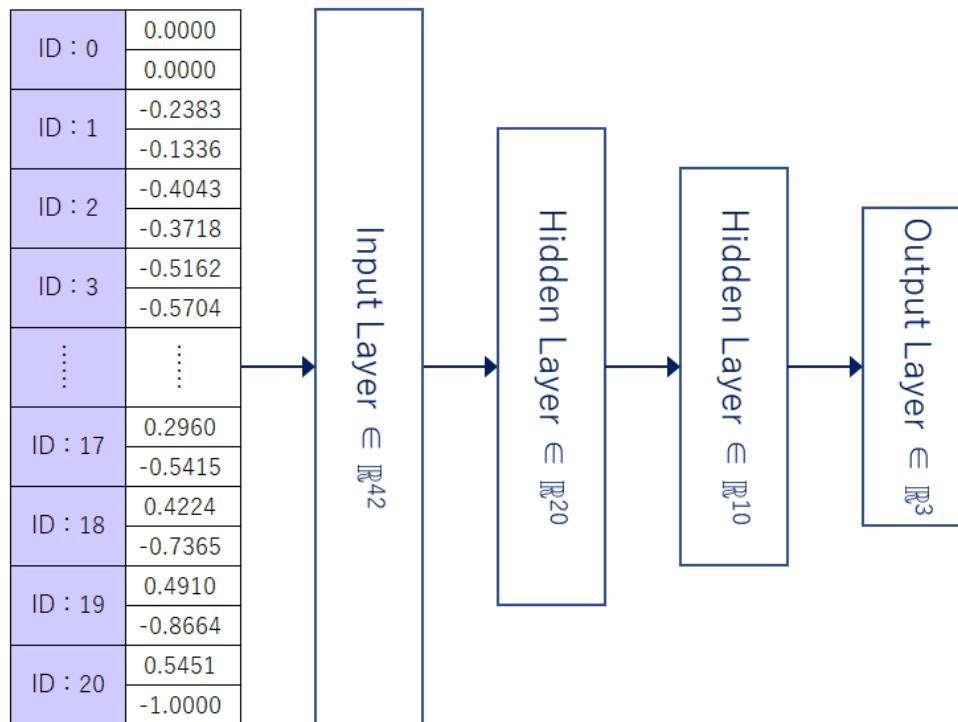


Figure 7.2 – Architecture du MLP : 42 entrées, 2 couches cachées (20 et 10 neurones), 4 sorties.

4.3 Dataset et Entraînement

Le modèle a été entraîné sur un dataset personnalisé (`keypoint.csv`) contenant **4787 échantillons** de coordonnées de landmarks. Ces données ont été collectées manuellement en enregistrant les gestes de la main du développeur dans différentes positions et orientations.

Paramètres d'entraînement :

- **Optimiseur** : Adam.
- **Fonction de perte** : Sparse Categorical Crossentropy.
- **Early Stopping** : Patience de 20 époques.

5 Pipeline Complet

Le flux de traitement pour chaque frame vidéo :

1. **Capture** : Le frontend capture une frame de la webcam.
2. **Transmission WebSocket** : L'image est envoyée au backend Python.
3. **MediaPipe Hands** : Extraction des 21 landmarks.
4. **Normalisation** : Conversion en coordonnées relatives au poignet.
5. **Classification MLP** : Prédiction du geste via le modèle TFLite.
6. **Retour WebSocket** : Le geste détecté est renvoyé au frontend.
7. **Action de jeu** : Le canvas React exécute l'action correspondante.

6 Implémentation Frontend

6.1 Composants React

L'interface du jeu est construite autour de plusieurs composants :

DinoGameCanvas.tsx Canvas HTML5 gérant le rendu du jeu (dinosauire, obstacles, score). Ce composant s'appuie sur la bibliothèque **react-chrome-dino** (v0.1.3), un portage React du jeu Chrome Dino original, permettant une intégration fluide dans l'écosystème React.

GestureDetector.tsx Composant capturant la webcam et communiquant avec le backend.

EducationalTutorial.tsx Tutoriel interactif expliquant les gestes au joueur.

Source : <https://www.npmjs.com/package/react-chrome-dino>

6.2 Communication WebSocket

La communication temps réel utilise WebSocket pour minimiser la latence :

`ws://localhost:8000/ws/gesture`

Le client envoie les frames compressées, le serveur répond avec le geste détecté.

7 Performances et Résultats

Les tests en conditions réelles ont démontré :

- **Latence** : < 50 ms bout-en-bout, imperceptible pour le joueur.
- **Précision** : Reconnaissance correcte de 95%+ des gestes en conditions normales.
- **Robustesse** : Détection stable malgré des variations d'éclairage modérées.

8 Vulgarisation

8.1 Tutoriel Interactif

Le composant **EducationalTutorial.tsx** propose une séquence pédagogique :

1. Démonstration des gestes attendus (animations).
2. Visualisation en temps réel des landmarks détectés.
3. Feedback immédiat sur le geste reconnu.

8.2 Visualisation des Landmarks

L'overlay affiche les 21 points clés sur la main du joueur, permettant de comprendre ce que "voit" l'IA.

Chapitre 8

Bilan et Perspectives

Ce chapitre final dresse le bilan du projet, synthétise les réalisations accomplies, les compétences acquises, les difficultés surmontées, et propose des pistes d'amélioration pour de futurs développements.

1 Synthèse des Réalisations

1.1 Rappel des Objectifs

L'objectif initial du projet était de concevoir un démonstrateur ludique et pédagogique exploitant la vision par ordinateur. Ce démonstrateur devait :

- Illustrer les capacités des modèles de détection d'objets (YOLO).
- Proposer une interaction homme-machine naturelle via la reconnaissance gestuelle.
- Vulgariser les concepts d'intelligence artificielle de manière accessible au grand public.

1.2 Bilan des Jeux Développés

Le tableau 8.1 synthétise l'état final des trois modules de jeu initialement prévus :

Jeu	Technologie	Statut	Raison
Où est Charlie ?	YOLOv8n	Abandonné	Dataset insuffisant, éclairage variable
Dobble	YOLOv8s	Livré	Réentraînement réussi avec augmentation
Dino Run	MediaPipe + MLP	Livré	Classifieur léger performant

TABLE 8.1 – Bilan des modules de jeu du démonstrateur.

1.3 Fonctionnalités Livrées

L'application finale intègre les fonctionnalités suivantes :

- **Détection en temps réel** : Analyse continue du flux webcam avec affichage des détections.
- **Modes de jeu variés** : IA seule, Joueur vs IA, tutoriel interactif.

- **Vulgarisation** : Séquences pédagogiques expliquant le fonctionnement des modèles.
- **Interface moderne** : Thème Cyberpunk avec support bilingue (français/anglais).
- **Déploiement conteneurisé** : Architecture Docker Compose pour une distribution portable.

2 Compétences Acquises

2.1 Lecture et Analyse de la Littérature Scientifique

Ce projet a été l'occasion d'approfondir la lecture d'articles scientifiques et de documentation technique. Cette compétence, essentielle pour tout ingénieur, a été développée à travers :

- **État de l'art** : Analyse comparative des architectures de détection (R-CNN, SSD, YOLO) et de leurs évolutions.
- **Documentation Ultralytics** : Compréhension des paramètres d'entraînement, techniques d'augmentation, et méthodes d'optimisation.
- **Articles MediaPipe** : Étude des pipelines de détection de main et d'extraction de landmarks.
- **Projets open-source** : Analyse du code source de projets existants (kinivi/hand-gesture-recognition-mediapipe) pour en extraire les bonnes pratiques.

2.2 Compétences Techniques

Vision par ordinateur : Entraînement de modèles YOLOv8, utilisation de MediaPipe, prétraitement d'images.

Machine Learning : Conception et entraînement de MLP, quantification TensorFlow Lite, data augmentation.

Développement Full-Stack : React/TypeScript (frontend), FastAPI/Python (backend), WebSocket temps réel.

Déploiement : Docker, Docker Compose, Nginx, gestion des dépendances.

2.3 Compétences Méthodologiques

- **Gestion de projet** : Planification avec Gantt, suivi via GitHub, documentation continue.
- **Adaptabilité** : Capacité à pivoter face aux difficultés (abandon de Charlie, introduction de Dino Run).
- **Communication** : Réunions régulières avec l'encadrant, rédaction de documentation technique.

3 Difficultés Rencontrées et Solutions

Le projet a été jalonné de plusieurs défis techniques, chacun ayant conduit à des adaptations stratégiques :

- **Jeu Charlie – Dataset insuffisant**. Le dataset de 570 images s'est avéré trop limité pour généraliser correctement. Les conditions d'éclairage variables lors des tests ont révélé un écart significatif entre l'entraînement et l'inférence réelle. *Solution* : Décision de pivot lors de la réunion du 12 novembre 2025, redirection des efforts vers Dino Run.

- **Jeu Dobble – Entraînement interrompu.** La contrainte des horaires de la salle informatique a conduit à un modèle sous-entraîné (80/300 époques). *Solution* : Demande d'accès aux serveurs GPU via la DSI, optimisation de la commande d'entraînement avec data augmentation.
- **Jeu Dino Run – Latence MediaPipe natif.** Le module de reconnaissance de gestes natif introduisait une latence incompatible avec le gameplay temps réel. *Solution* : Adoption d'une approche hybride inspirée du projet kinivi, avec un classifieur MLP léger (1114 paramètres) offrant une inférence quasi-instantanée.

4 Perspectives

4.1 Amélioration du Jeu Charlie

Le jeu "Où est Charlie?" pourrait être réactivé en appliquant les enseignements tirés du succès de Dobble :

- **Augmentation du dataset** : Collecter ou générer davantage d'images annotées (objectif : 2000+ images).
- **Data augmentation agressive** : Utiliser la méthode `tune()` d'Ultralytics pour optimiser automatiquement les hyperparamètres, incluant les paramètres d'augmentation :

```
from ultralytics import YOLO

model = YOLO("yolov8s.pt")
model.tune(
    data="data_charlie.yaml",
    epochs=30,
    iterations=100,
    optimizer="AdamW",
)
```

Cette approche permettrait de simuler des conditions d'éclairage variées et d'améliorer la robustesse du modèle en conditions réelles.

5 Conclusion Personnelle

Ce projet de recherche et innovation a constitué une expérience formatrice à plusieurs égards. Au-delà des compétences techniques acquises en vision par ordinateur et développement full-stack, il m'a permis de développer une méthodologie de travail rigoureuse face à des problématiques ouvertes.

La nécessité de pivoter en cours de projet, passant du jeu Charlie à Dino Run, a été un apprentissage précieux sur l'importance de l'adaptabilité et de la prise de décision éclairée. Cette expérience m'a également sensibilisé à l'écart fréquent entre les performances théoriques d'un modèle et son comportement en conditions réelles.

Enfin, la dimension pédagogique du projet, visant à vulgariser les concepts d'intelligence artificielle, m'a amené à réfléchir sur la transmission des savoirs techniques au grand public, une compétence de plus en plus valorisée dans le contexte actuel de démocratisation de l'IA.

Conclusion

Ce projet de recherche et innovation a permis de développer un démonstrateur complet de vision par ordinateur, illustrant de manière ludique et pédagogique les capacités des technologies modernes d'intelligence artificielle.

Bilan Technique

Les deux jeux implémentés démontrent des applications concrètes de la vision par ordinateur :

- **Dobble** exploite YOLOv8 pour la détection d'objets en temps réel, avec un modèle réentraîné sur un dataset personnalisé de symboles. Les modes compétitif et pédagogique permettent de vulgariser le fonctionnement des réseaux de neurones.
- **Dino Run** combine MediaPipe pour l'extraction de landmarks de la main avec un perceptron multicouche léger pour la classification des gestes. Cette approche hybride offre une latence suffisamment faible pour le contrôle temps réel d'un jeu.

L'architecture entièrement conteneurisée via Docker garantit la portabilité et la reproductibilité du déploiement, facilitant les démonstrations sur diverses configurations matérielles.

Enseignements

Ce projet a été l'occasion d'approfondir de nombreuses compétences techniques : entraînement de modèles de deep learning, développement full-stack, conteneurisation, et gestion de projet. La confrontation aux contraintes du temps réel et à la variabilité des conditions d'utilisation a également permis de développer une approche pragmatique de résolution de problèmes.

Perspectives

Le démonstrateur constitue une base solide pour de futures évolutions : ajout de nouveaux jeux, amélioration des modèles IA. La structure modulaire du code facilite l'intégration de ces améliorations.

Bibliographie

- [1] Juan Terven et Diana Cordova-Esparza. « A Comprehensive Review of YOLO Architectures in Computer Vision : From YOLOv1 to YOLOv8 and YOLO-NAS ». en. In : Machine Learning and Knowledge Extraction 5.4 (nov. 2023). arXiv :2304.00501 [cs], p. 1680-1716. issn : 2504-4990. doi : 10.3390/make5040083. url : <http://arxiv.org/abs/2304.00501>.
- [2] Jyotishman Bora et al. « Real-time Assamese Sign Language Recognition using MediaPipe and Deep Learning ». In : Procedia Computer Science. International Conference on Machine Learning and Data Engineering 218 (jan. 2023), p. 1384-1393. issn : 1877-0509. doi : 10.1016/j.procs.2023.01.117. url : <https://www.sciencedirect.com/science/article/pii/S1877050923001175>.
- [3] « A Study on Real-time Hand Gesture Recognition Technology by Machine Learning-based MediaPipe ». en. In : Journal of System and Management Sciences (avr. 2022). issn : 18166075, 18180523. doi : 10.33168/JSMS.2022.0225. url : <http://www.aasmr.org/jsms/Vol12/JSMS%20April%202022/Vol.12No.02.25.pdf>.
- [4] Marius-Constantin Popescu et al. « Multilayer Perceptron and Neural Networks ». en. In : ().

Annexes

Guide de Déploiement

Ce guide détaille les étapes nécessaires pour déployer l'application sur une machine vierge.

Prérequis

Logiciels Requis

- **Docker Desktop** : Version 24.0 ou supérieure.
 - Windows : Télécharger depuis <https://www.docker.com/products/docker-desktop/>
 - Activer WSL 2 (Windows Subsystem for Linux) si demandé lors de l'installation.
- **Git** : Pour cloner le dépôt (optionnel si le code est fourni en archive).

Configuration Matérielle Recommandée

- **RAM** : 8 Go minimum (16 Go recommandé).
- **Espace disque** : 5 Go disponibles pour les images Docker.
- **Webcam** : Requête pour les fonctionnalités de reconnaissance.
- **Navigateur** : Chrome, Firefox ou Edge (version récente avec support WebRTC).

Étapes de Déploiement

Étape 1 : Récupération du Code Source

```
git clone https://github.com/Isopope/ComputerVisionPRI25-26.git
cd ComputerVisionPRI25-26
```

Ou extraire l'archive ZIP fournie :

```
unzip ComputerVisionPRI25-26.zip
cd ComputerVisionPRI25-26
```

Étape 2 : Lancement de Docker Desktop

S'assurer que Docker Desktop est démarré et fonctionnel. Vérifier avec :

```
docker --version
docker-compose --version
```

Étape 3 : Construction et Démarrage des Conteneurs

Depuis la racine du projet, exécuter :

```
docker-compose up --build
```

Cette commande :

1. Construit l'image Docker du backend (Python/FastAPI).
2. Construit l'image Docker du frontend (React compilé + Nginx).
3. Démarre les deux conteneurs en réseau.

Note : La première construction peut prendre 40 à 50 minutes selon la connexion Internet (téléchargement des dépendances Python et npm).

Étape 4 : Accès à l'Application

Une fois les conteneurs démarrés, ouvrir un navigateur et accéder à :

`http://localhost:8080`

Étape 5 : Autorisation de la Webcam

Lors du premier accès, le navigateur demandera l'autorisation d'utiliser la webcam. Cliquer sur **Autoriser** pour activer les fonctionnalités de reconnaissance.

Commandes Utiles

Commande	Description
<code>docker-compose up -d</code>	Démarre en arrière-plan.
<code>docker-compose down</code>	Arrête et supprime les conteneurs.
<code>docker-compose logs -f</code>	Affiche les logs en temps réel.
<code>docker-compose build --no-cache</code>	Reconstruction complète sans cache.
<code>docker ps -a</code>	Liste tous les conteneurs (actifs et arrêtés).

TABLE 8.2 – Commandes Docker Compose utiles.

Dépannage

Erreur "Port 8080 already in use"

Un autre service utilise déjà le port 8080. Solutions :

- Arrêter le service conflictuel.
- Modifier le port dans `docker-compose.yml` (ex : 8081:80).

La webcam ne fonctionne pas

- Vérifier que le navigateur a l'autorisation d'accéder à la caméra.
- Utiliser HTTPS si nécessaire (certains navigateurs bloquent WebRTC en HTTP pour des raisons de sécurité).
- Tester avec `chrome://settings/content/camera` pour Chrome.

Les conteneurs ne démarrent pas

- Vérifier que Docker Desktop est bien lancé.
- Consulter les logs : `docker-compose logs`.
- S'assurer d'avoir assez d'espace disque.

Mode Développement (Optionnel)

Pour contribuer au développement sans Docker :

Backend

```
cd backend
python -m venv .venv
.venv\Scripts\activate      # Windows
pip install -r requirements.txt
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev
```

L'application sera accessible sur `http://localhost:5173` (Vite dev server).

Captures d'Écran de l'Application

Cette annexe présente les principales interfaces de l'application finale.

Page d'Accueil

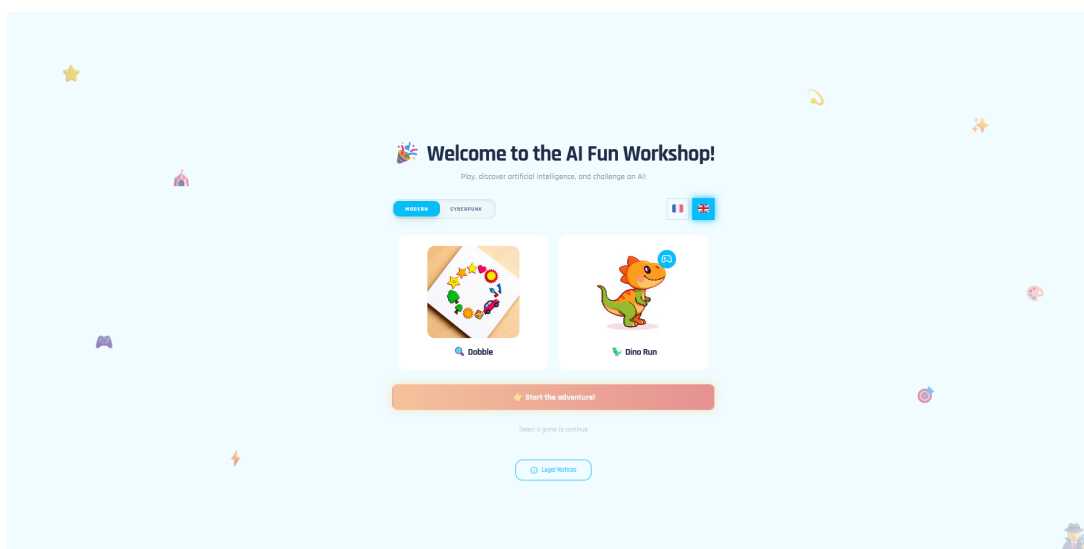


Figure 8.1 – Page d'accueil de l'application.

Jeu Dobble

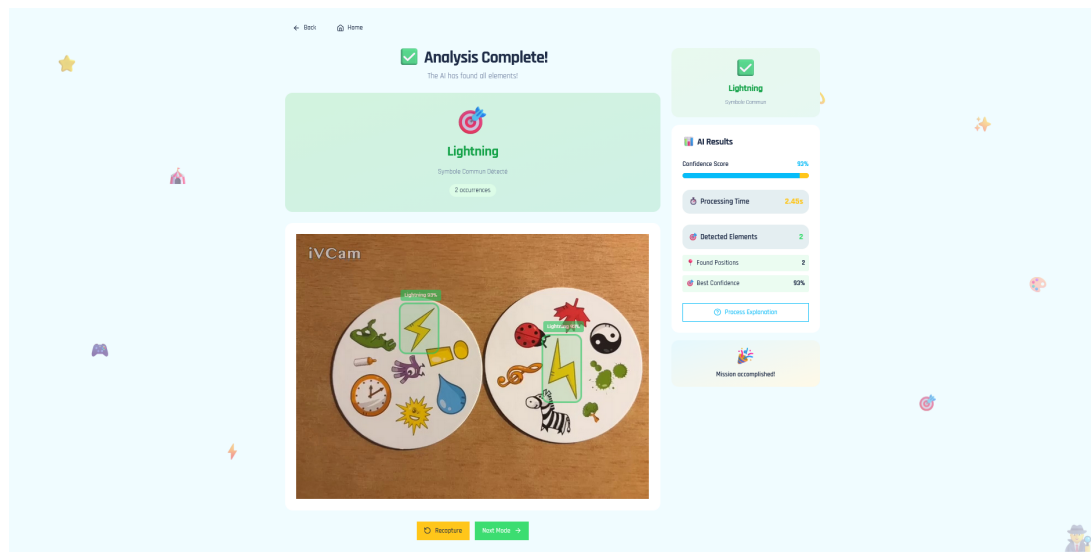


Figure 8.2 – Jeu Dobble : détection des symboles en temps réel.

Mode Explicatif Dobble

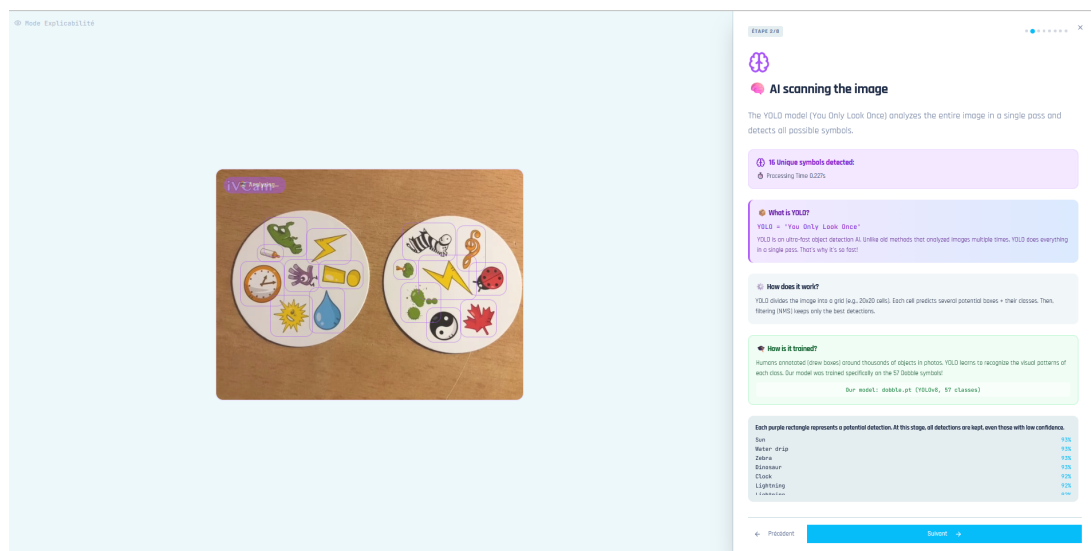


Figure 8.3 – Mode explicatif Dobble : visualisation du fonctionnement de YOLOv8.

Jeu Dino Run

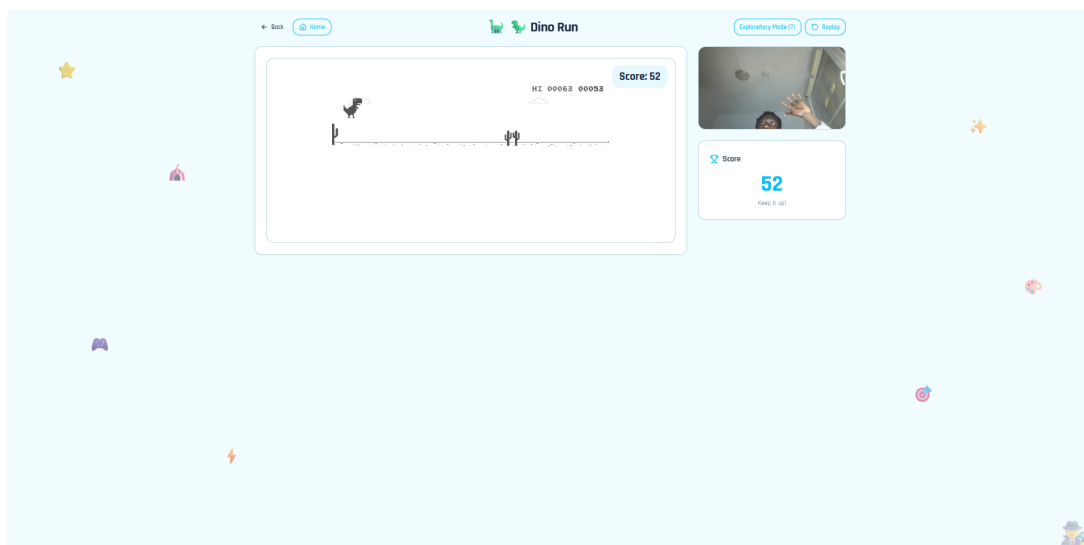


Figure 8.4 – Jeu Dino Run : contrôle gestuel du dinosaure.

Mode Explicatif Dino Run

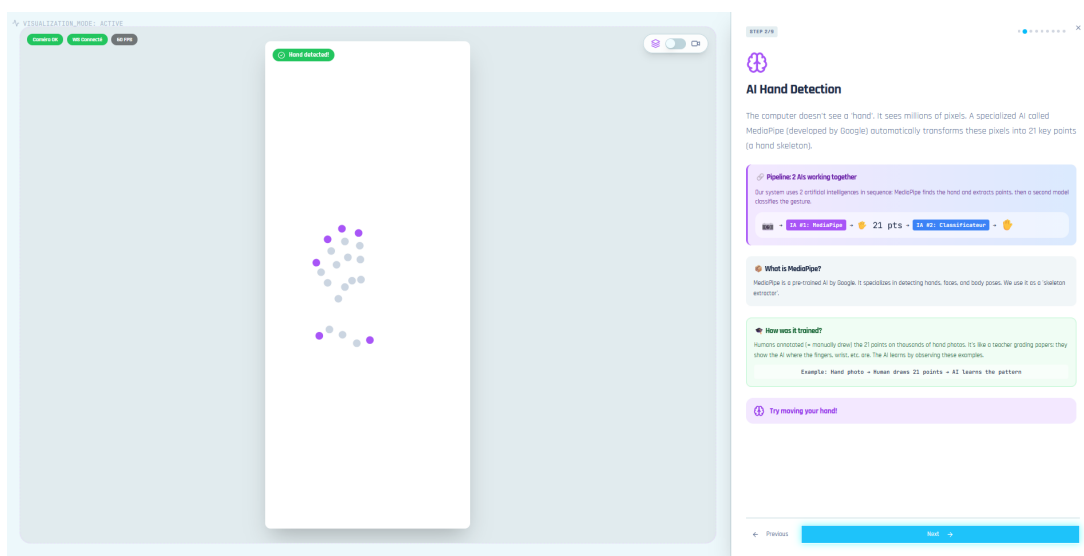


Figure 8.5 – Tutoriel interactif Dino Run : visualisation des landmarks de la main.

Comparaison des Thèmes

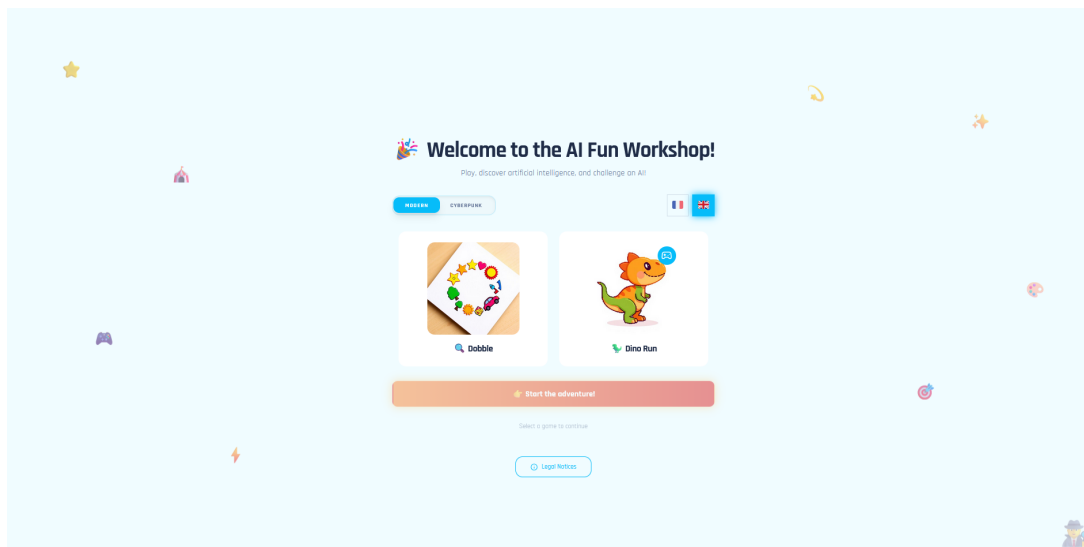


Figure 8.6 – Thème Classique.

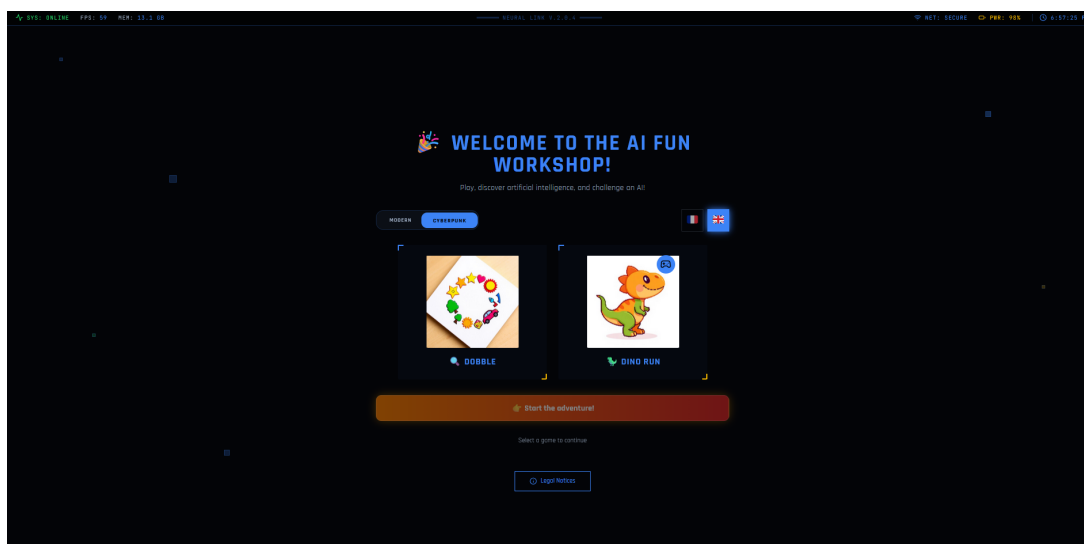


Figure 8.7 – Thème Cyberpunk.

Démonstrateur de Computer Vision : Jeux Dobble et Dino Run

Résumé :

Ce projet de recherche et innovation porte sur le développement d'un démonstrateur de vision par ordinateur intégrant deux jeux interactifs. Le premier, inspiré du jeu Dobble, utilise YOLOv8 pour la détection de symboles en temps réel. Le second, basé sur le célèbre jeu Chrome Dino, exploite MediaPipe et un perceptron multicouche pour la reconnaissance de gestes de la main. L'application web, construite avec React et FastAPI, est entièrement conteneurisée via Docker pour garantir la portabilité. Ce démonstrateur illustre les applications pratiques de l'intelligence artificielle dans le domaine du jeu vidéo et de l'interaction homme-machine.

Mots clés :

Vision par ordinateur, YOLOv8, MediaPipe, Détection d'objets, Reconnaissance de gestes, React, FastAPI, Docker, Intelligence artificielle, Temps réel.

Abstract :

This research and innovation project focuses on developing a computer vision demonstrator featuring two interactive games. The first, inspired by the Dobble game, uses YOLOv8 for real-time symbol detection. The second, based on the famous Chrome Dino game, leverages MediaPipe and a multilayer perceptron for hand gesture recognition. The web application, built with React and FastAPI, is fully containerized using Docker to ensure portability. This demonstrator showcases practical applications of artificial intelligence in gaming and human-computer interaction.

Keywords :

Computer Vision, YOLOv8, MediaPipe, Object Detection, Gesture Recognition, React, FastAPI, Docker, Artificial Intelligence, Real-time.